

Τεχνητή νοημοσύνη εργασία 1

Αλέξανδρος Μακρυγιάννης : p3210271
Μάριος Δημήτριος Καπότης: p3220065
Ελισάβετ Μαρία Μπινετοπούλου: p3220132

A.

Στο συγκεκριμένο θέμα αποφασίσαμε να υλοποιήσουμε το υποερώτημα B. Υλοποιήσαμε το κωδικά με την μεθοδο του γενετικού αλγορίθμου.

Οι περιορισμοί που πήραμε ήταν οι εξής:

- Όλες οι 9 τάξεις να κανουν μαθημα ακριβως 35 ωρες την εβδομαδα, χωρις κενά μέσα στην μέρα.
- Να μην ολοκληρώνονται οι ωρες του μαθήματος σε 1 ημέρα (εκτός και αν προκειται για ενα μαθημα το οποιο διδασκεται μια ωρα την εβδομαδα)
- Οι δάσκαλοι να μην διδάσκουν 3 ωρες (ή παραπάνω) συνεχόμενα.

Κάποιοι βασικοί περιορισμοι που το προγραμμα αναγκαστικά τηρει:

- Ενας καθηγητης δεν μπορει να διδάσκει σε 2 (ή παραπάνω) τάξεις συνεχόμενα.
- Τα τηρούνται τα daily hours και τα weekly hours που εχει δηλώσει ενας καθηγητης καθώς και να ολοκληρώνονται η ώρες του μαθήματος σε καθε τάξη

Βέβαια, με αυτούς τους περιορισμούς που λάβαμε ηρθε και με ενα κόστος. Το πρόγραμμα δυσκολευεται να βρει ενα καταλληλο schedule που να μην παραβιάζει κανέναν από αυτούς τους περιορισμούς (που να έχει fitness = 0). Όσο πιο μικρο είναι το fitness που δίνεται τόσο μεγαλύτερο είναι το runtime του προγράμματος, αλλά δουλεύει.

Περιγραφή των κλάσεων

Καταρχάς τα αρχεία που θα δέχεται το πρόγραμμα πρέπει να έχουν αυτήν την μορφή:

teachers.txt

Κωδικός / Ονομα / κωδικός μαθήματος, κωδικος μαθήματος,.....,κωδικός μαθηματος /
Μέγιστος Αριθμός Ωρων ανα ημερα / Μέγιστος αριθμός ωρων ανα εβδομάδα

lesson.txt

Κωδικός / Ονομα / Τάξη / Μέγιστος αριθμος ωρών που πρεπει να διδάσκεται το μαθημα

→ Κλάση Lesson:

Σε αυτήν την κλάση, μπαίνουν τα δεδομένα που παιρνουμε απο το αρχειο lessons.txt σε συγκεκριμένες μεταβλητες. Εχουμε βάλει και μια επιπλεον μεταβλητη που ονομάζεται Teacher και είναι κλασης Teacher (βλεπε παρακάτω).

→Κλάση Teacher:

Σε αυτήν την κλάση, όπως και στο lesson.txt, μπαίνουν τα δεδομένα που παιρνουμε απο το αρχειο teachers.txt. Τα μαθήματα που διδάσκει ο καθηγητης θα ειναι ενα μονοδιάστατος πίνακας κλάσης Lessons που θα μπαίνουν τα μαθήματα που διδάσκει ο κάθε καθηγητής.

→ Κλάση Subject:

Σε αυτήν την κλάση, υπάρχουν 2 μεταβλητές, η codeLesson και η codeTeacher όπου είναι κλάσεις String.

→ Κλάση Main

Μέθοδοι:

- Μέθοδος parseLessons: Παίρνει το αρχείο "lessons.txt" και με βάση τα δεδομένα, φτιάχνει μεταβλητές τύπου Lesson και τις κάνει store σε ένα ArrayList τύπου Lesson. Επιστρέφει στο κύριο πρόγραμμα το ArrayList αυτό.
- Μέθοδος parseTeachers: Παιρνει το αρχείο teachers.txt και με βάση τα δεδομένα, φτιάχνει μεταβλητές τύπου Teacher (με την βοήθεια της μεθόδους getLessons παίρνει τους κωδικούς των μαθημάτων και βρίσκει τα καταλληλα μαθήματα από το ArrayList lessons) και τις κάνει store σε ένα ArrayList τύπου Teacher.
- Μέθοδος createSchedule: Παίρνει το Chromosome που βρήκε το GeneticAlgorithm, το ArrayList lessons και δημιουργεί ένα String.
- Μέθοδος getLessons: Η μέθοδος αυτή παίρνει την τάξη και με αυτήν βρίσκει καταλληλα μαθηματα που αντιστοιχουν σε αυτήν. Για κάθε μάθημα που ανήκει στην τάξη, το προσθέτει στη λίστα newLessons τόσες φορές, όσες ώρες διδάσκεται την εβδομάδα. Τελος επιστρέφει το ArrayList newLessons

→ Κλάση Chromosome

Μέθοδοι:

Chromosome: Δημιουργεί έναν πίνακα διαστάσεων 9x35 (9 τάξεις και 35 ημέρες) και με βάση την κάθε τάξη παίρνει τα καταλληλα μαθηματα και τα βάζει σε τυχαία σειρά.

Chromosome(για αντιγραφές): Παιρνει το genes που περάστηκε και το αντιγράφει στο this.genes.

calculateFitness: Βρίσκει τους περιορισμούς που παραβιάζει ο πίνακας this.genes

mutate: Διαλέγει τυχαία μια τάξη από τις 9 (από 0 έως 8) και παίρνει 2 τυχαίες ώρες της εβδομάδας (από 0 έως 34) και τις αντιμεταθέτει.

→ Κλάση GeneticAlgorithm

Μέθοδοι:

run: Εκτελεί τον αλγόριθμο και επιστρέφει το καλύτερο χρωμόσωμα.

initializePopulation: Δημιουργεί τον αρχικό πληθυσμό.

updateOccurences: Καθορίζει τις πιθανότητες επιλογής γονέων.

reproduce: Δημιουργεί νέα χρωμοσώματα από δύο γονείς.

B.

B1)Υλοποίηση με την μέθοδο Forward Chaining με Java:

Βάση Γνώσης(BΓ):

Αποτελείται από δύο πράγματα

-Τα **Facts** (γεγονότα) που είναι πληροφορίες που θεωρούνται αληθείς και δηλώνονται στην βάση με το πρόθεμα "**FACT:**"

-Τα **Rules**(κανόνες). Οι κανόνες συνδέουν τα γεγονότα με τα συμπεράσματα. Κάθε κανόνας αποτελείται από:

- **Προϋποθέσεις** : Στο πρώτο μέρος της σχέσης, τα γεγονότα που πρέπει να ισχύουν για να ενεργοποιηθεί ο κανόνας.
- **Συμπέρασμα**: Στο δεύτερο μέρος της σχέσης, το νέο γεγονός που μπορεί να εξαχθεί αν ικανοποιούνται οι προϋποθέσεις.

Τα Rules δηλώνονται στην βάση γνώσης με το πρόθεμα "**RULE:**"

παραδείγματα Rule :

RULE: A => C

RULE: C => D

Κλάση main:

Φορτώνει αυτόματα το αρχείο "**knowledge_base.txt**" το οποίο αποτελεί την βάση γνώσης του προγράμματος μας, σε περίπτωση που προκύψει κάποιο λάθος με την φόρτωση του αρχείου πετάει exception με μήνυμα ότι υπάρχει πρόβλημα με το αρχείο. Στην συνέχεια ζητάει από τον χρήστη να πληκτρολογήσει έναν τύπο (query) το οποίο θέλει να αποδείξει. Για να το πετύχει , οργανώνει τα rules μέσω της **loadKnowledgeBase(FileKnowledge, facts)** που έχει ως μεταβλητές την βάση γνώσης και την list facts και έπειτα πηγαίνει στην **performForwardChaining(rules, facts, query)** με μεταβλητές τα rules και τα facts της ΒΓ και το query και επιστρέφει την λύση της απόδειξης (αν είναι αληθής ή ψευδής) το οποίο εκχωρείται στην μεταβλητή result. Τέλος εκτυπώνει ανάλογο μήνυμα ανάλογα αν είναι αληθής ή ψευδής ο τύπος.

Κλάση Rule:

Η κλάση Rule χρησιμοποιείται για την αναπαράσταση ενός κανόνα και χρησιμοποιείται για την παραγωγή και για την αποθήκευση του .έχει τα πεδία conditions δηλαδή μια λίστα από προϋποθέσεις (αριστερό μέρος του Rule) και το conclusion δηλαδή το συμπέρασμα (δεξί μέρος του Rule). Ως μεθόδους έχει το getConditions που επιστρέφει τις προϋποθέσεις του κανόνα και το getConclusion που επιστρέφει το συμπέρασμα του κανόνα.

Μέθοδος loadKnowledgeBase(String fileName, List<String> facts):

Όπως προαναφέρθηκε, δέχεται ως είσοδο της βάση γνώσης και έχει έξοδο την λίστα με τα rules και την λίστα με τα facts. Διαβάζει με το BufferedReader το έγγραφο και:

- Άμα ξεκινάει η γραμμή με **"FACT:"** τότε προσθέσει το γεγονός στην λίστα γεγονότων
- Αλλιώς άμα ξεκινάει με το **"RULE:"** τότε αναλύει τον κανόνα στις προϋποθέσεις και το συμπέρασμα το προσθέτει στη λίστα των κανόνων

Μέθοδος performForwardChaining(List<Rule> rules, List<String> facts, String query):

Αυτή η μέθοδος εκτελεί την προωθητική αλυσιδωτή συμπερασματολογία και ως είσοδο έχει την λίστα με τα rules, την λίστα με τα facts και το ερώτημα (query). Όσο η ουρά του ChainingQueue είναι άδεια ελέγχεται αν το γεγονός είναι ίδιο με το ερώτημα (query). Αν ναι, επιστρέφει **true**. Επίσης για κάθε κανόνα, αν οι προϋποθέσεις του κανόνα ικανοποιούνται από τα γνωστά γεγονότα, τότε το συμπέρασμα του κανόνα προστίθεται στα γεγονότα και στην ουρά. Ειδάλλως αν εξαντληθούν όλα τα γεγονότα και δεν αποδειχθεί το ερώτημα, επιστρέφει **false**.

Παράδειγμα Χρήστη :

Έστω η παρακάτω **βάση γνώσης** στο αρχείο **knowledge_base.txt**:

FACT: A

FACT: B

RULE: A => C

RULE: C => D

Ως γεγονότα έχουμε το A και το B και ο χρήστης θέλει να αποδείξει το D.

στην Βάση γνώσης ξέρουμε ότι ισχύουν τα ως κανόνες

- **A => C** ισχύει γιατί το **A** είναι γνωστό. Άρα προσθέτουμε το **C** στα γεγονότα
- **C => D** ισχύει γιατί το **C** μόλις αποδείχθηκε. Άρα προσθέτουμε το **D** στα γεγονότα.

Άρα το γεγονός D είναι γνωστό και αρα το πρόγραμμα επιστρέφει:

```

C:\Users\itzpr\Desktop\B1>javac ForwardChaining.java

C:\Users\itzpr\Desktop\B1>java ForwardChaining
Fortosi tis vaseis gnoseos apo to arxeio : knowledge_base.txt
...
Dose ton pros apodeixi typo:
D
Alithes, O typos D apodeixthike.

C:\Users\itzpr\Desktop\B1>

```

B3)

Υλοποίηση μεθόδου Backward Chaining με Java:

-> Κλάση *BackwardChaining*:

- Μέθοδος `loadFile(String FileName)`: φορτώνεται το αρχείο με τα Facts, τους κανόνες και τα συμπεράσματα όπου έχει βελάκια. Η μέθοδος τα διαβάζει και τα βάζει στις κατάλληλες λίστες.
- Μέθοδος `backwardChaining(String query)`: φορτώνεται με το ζήτημα, το query, που πρέπει να ελέγξει. Αν το query είναι ήδη γνωστο επιστρέφει αμέσως true. Αλλιώς κάνει τους κατάλληλους ελέγχους και επιστρέφει true η false.
- Μέθοδος `main()`: Φτιάχνει ένα αντικείμενο κλάσης *BackwardChaining*, φορτώνει το `arxeio.txt` και ζητάει από τον χρήστη να δώσει ένα ζητούμενο (query). Καλεί τις κατάλληλες μεθόδους και κάνει print τα αποτελέσματα.

->Κλάση *Rule*:

- Εκεί υπάρχει η λίστα με τις προϋποθέσεις και το συμπέρασμα κάθε κανόνα. (Φτιάχτηκε για την διευκόλυνση στην αναγνώση και αποθηκευση των κανονων σε λίστες με τύπο *Rule*).

Δυνατότητες Λογισμικού:

- ❖ Υποστηρίζει την ανάγνωση αρχείων που περιέχουν γεγονότα και κανόνες.
 - Γεγονότα (Facts): Δηλώσεις που θεωρούνται αληθείς εξ αρχής.
 - Κανόνες (Rules): Σχέσεις μεταξύ συνθηκών και συμπερασμάτων, οι οποίες εκφράζονται στη μορφή $A_1, A_2, \dots, A_n$ και B το συμπέρασμα
- ❖ Χρησιμοποιεί αναδρομή για να ελέγξει αν το ζητούμενο, το query, μπορεί να αποδειχθεί βάση των γνωστών γεγονότων και κανόνων που δίνονται από το αρχείο.
- ❖ Επεξεργάζεται κάθε κανόνα για να εξετάσει αν το συμπέρασμα του ταιριάζει με το ζητούμενο, εξετάζοντας παράλληλα όλες τις συνθήκες.
- ❖ Δίνει τη δυνατότητα επέκτασης των κανόνων και γεγονότων επειδή δουλεύει με την χρήση ενός αρχείου το οποίο μπορεί να αλλάζει, διευκολύνοντας την προσαρμογή σε διαφορετικά σενάρια εφαρμογής.

Παράδειγμα Χρήσης:

-Ας πούμε ότι το αρχείο `arxeio.txt` είναι το παρακατω:

Fact: Sunny

Fact: Weekend
Sunny, Weekend -> GoToBeach
Fact: HaveCar
GoToBeach, HaveCar -> RoadTrip

Ας το αναλύσουμε. Είναι γεγονός ότι έχει ήλιο, είναι σαββατοκύριακο και ότι έχω αμάξι. Οι “κανόνες” μας είναι ότι αν έχει ήλιο και είναι σαββατοκύριακο, θα πάμε στην θάλασσα. Επίσης, αν πάμε στην θάλασσα και έχω αμάξι, θα πάμε εκδρομή. Δίνουμε το query στο λογισμικό, “RoadTrip”, δηλαδή το ζητούμενο μας είναι αν θα πάμε εκδρομή.

Το λογισμικό μας θα ελέγξει αρχικά αν η εκδρομή είναι μέσα στα Facts. Όμως, θα βρει τον κανόνα που περιγράψαμε παραπάνω και θα δει αν πληρούνται οι συνθήκες για να πάμε εκδρομή. Αφού θα πάμε στην θάλασσα, (Facts: Sunny, Weekend άρα ισχύει ότι GoToBeach). Επίσης είναι γεγονός ότι έχω αμάξι, αρα το λογισμικό θα επιστρέψει True. Πράγματι, όπως βλέπετε παρακάτω στο screenshot, ισχυει.

```
C:\Users\ellie\Downloads>javac BackwardChaining.java  
  
C:\Users\ellie\Downloads>java BackwardChaining  
Enter query: RoadTrip  
Result: true
```